LOCUS SOFTWARE FELLOWSHIP DAY-01

Frontend Fundamentals:
HTML, CSS & JavaScript

Kickstart your web development journey. Build and style your very first interactive landing page from scratch.

INSTRUCTOR
Sunil Lamichhane

Let's Get Started

Introduction to Frontend Development

Let’s set outcomes and expectations

What we won’t be doing today.?

- Building facebook
- Going too deep, into nooks and corners to web technologies
- Killing the instructor
- Talking about worldcup

What we will be doing today.?

- Build a cool portfolio website
- Have a overview and bird eye view into this world of web development
- Learn basics enough to get started
- Have Fun

Let’s set outcomes and expectations

There is always room to grow.

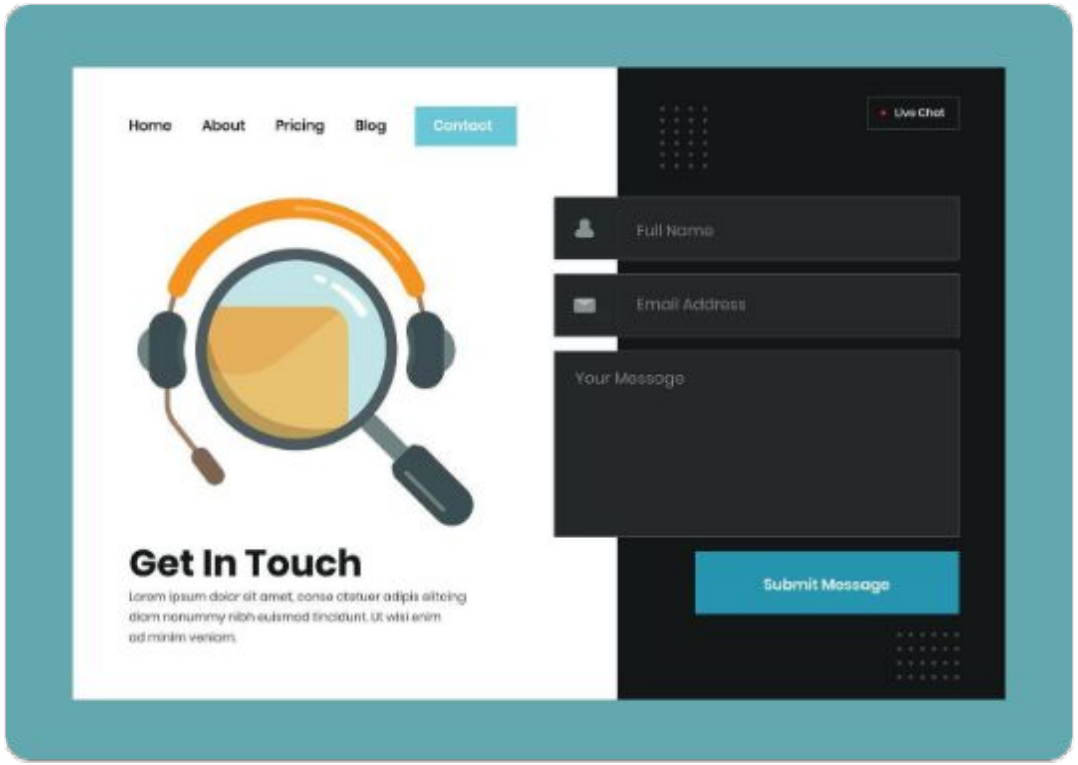
If what I’m teaching you today is something you already know or feels too basic for you, go for the extra mile, do something further in what we are doing. There is always some room to learn and grow.

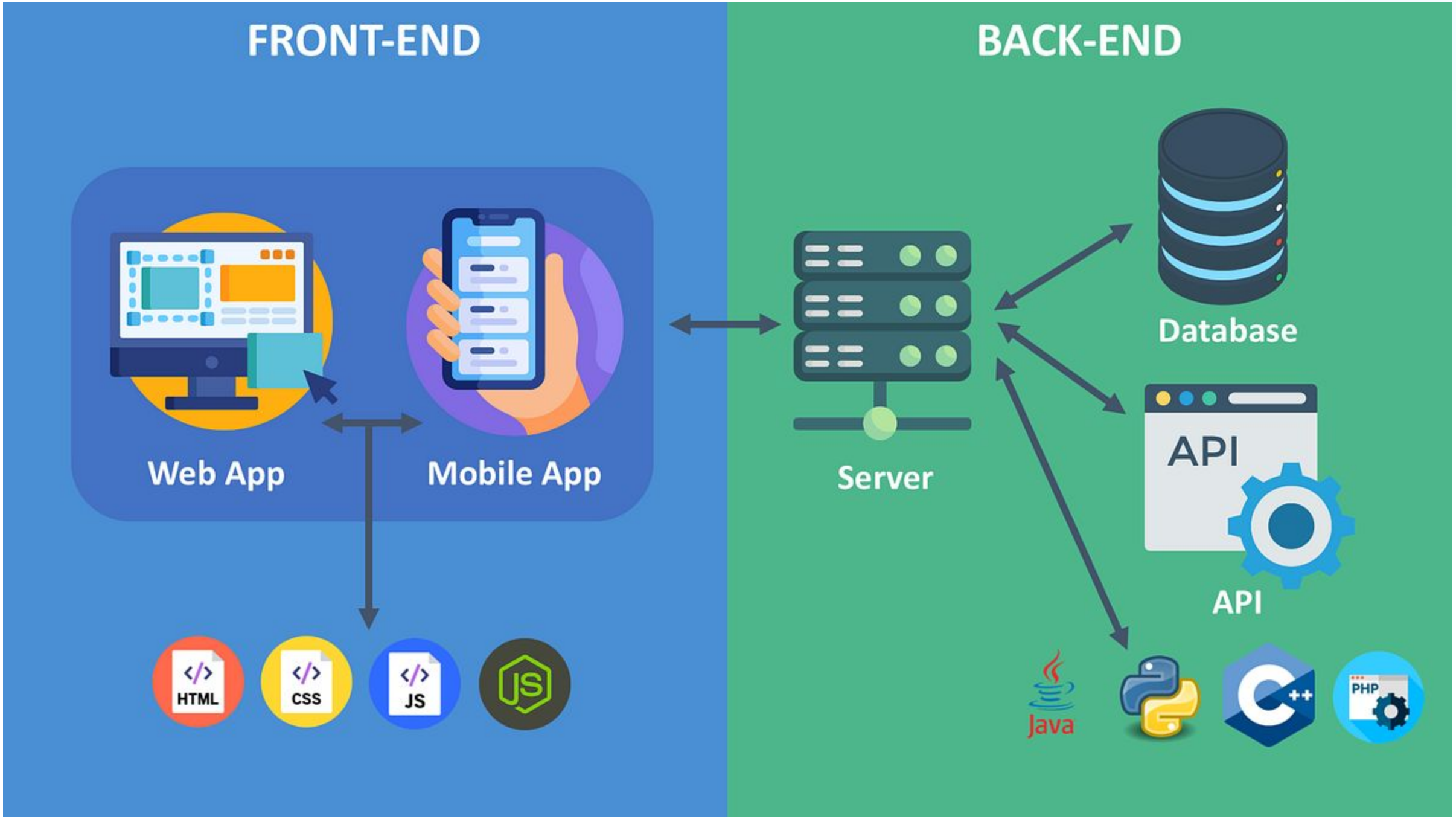
What is Web Development?

The process of building websites and web applications that run in a browser.

Frontend Development: Everything the user sees, clicks, and interacts with directly inside their browser window.

Backend Development: The background machinery — servers, databases, security, and logic that users never see.




FRONT-END
BACK-END
Web App
Mobile App
Server
Database
API
API
HTML
CSS
JS
Java
PHP

Quick QnA?

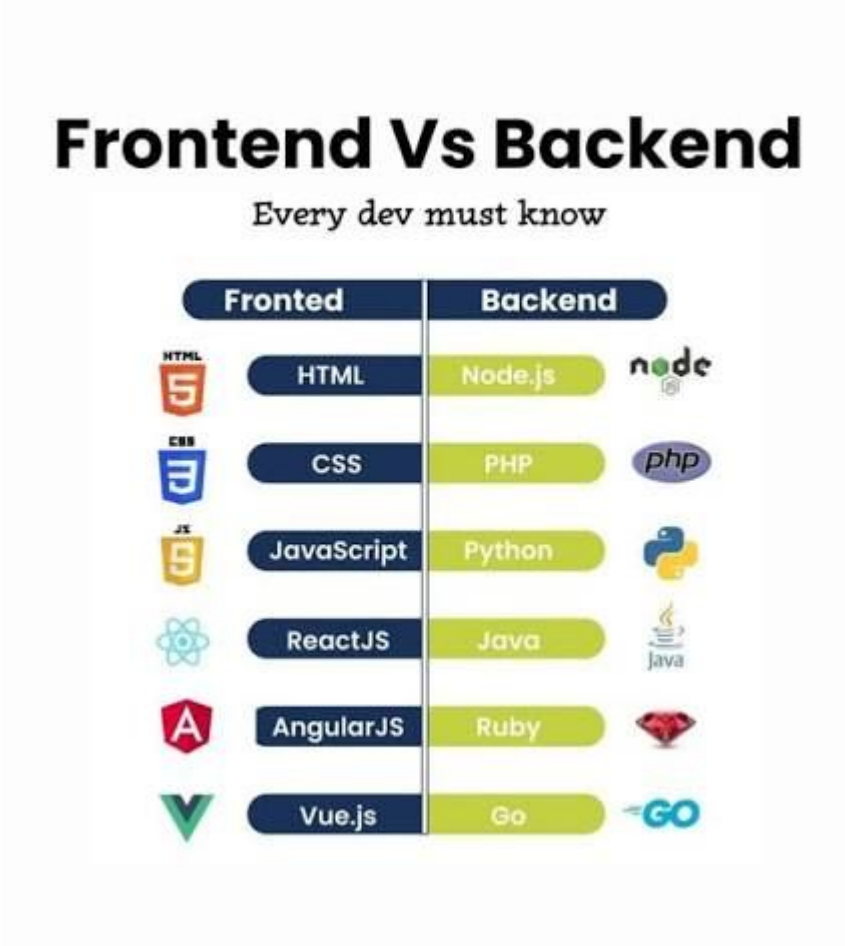
Can anybody name 5-5 different technologies for

Frontend and Backend?

Quick QnA?

Can anybody name 5-5 different technologies for

Frontend and Backend?



LOCUS 2027
23rd National Technological Festival

Data Hub

What is Frontend?

Everything a user sees and interacts with in the browser. It translates raw code into visual layouts, styled design patterns, smooth interfaces, and fluid behaviors.

THE CORE TECH STACK

HTML, CSS, JS — The Analogy

HTML

THE SKELETON

Defines the structure of your website — titles, paragraphs, images, buttons, and structured layout sections.

CSS

THE SKIN & CLOTHES

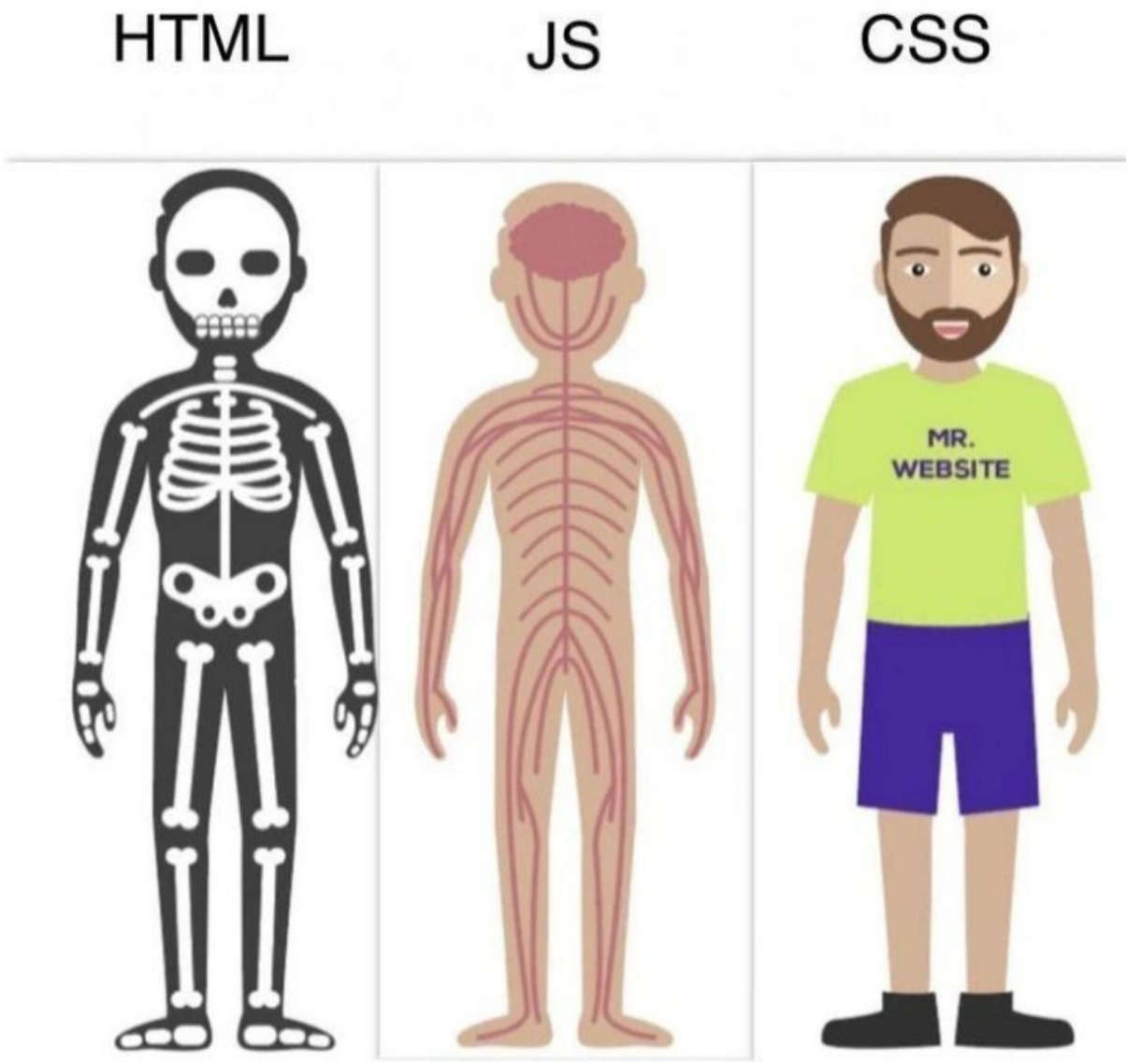
Brings color, fonts, spacings, and layouts to life. It makes the skeleton look appealing, uniform, and responsive.

JavaScript

THE MUSCLES

Powers functional interactions — showing warning popups, fetching live databases, submitted forms handling, and custom animations.

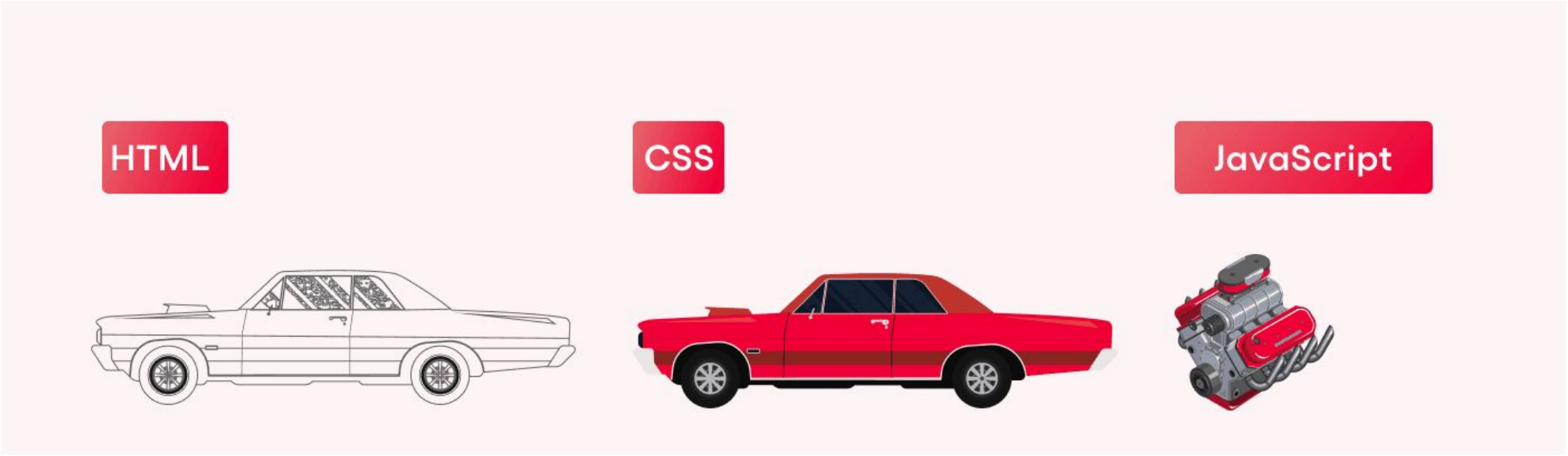
HTML, CSS, JS — The Analogy



LOCUS 2027
23rd National Technological Festival

Data Hub

HTML, CSS, JS — The Analogy



How Does a Browser Work?

The browser is an engine that processes your local files and transforms them on your computer screen. No magic servers or complex deployment tools are required yet.

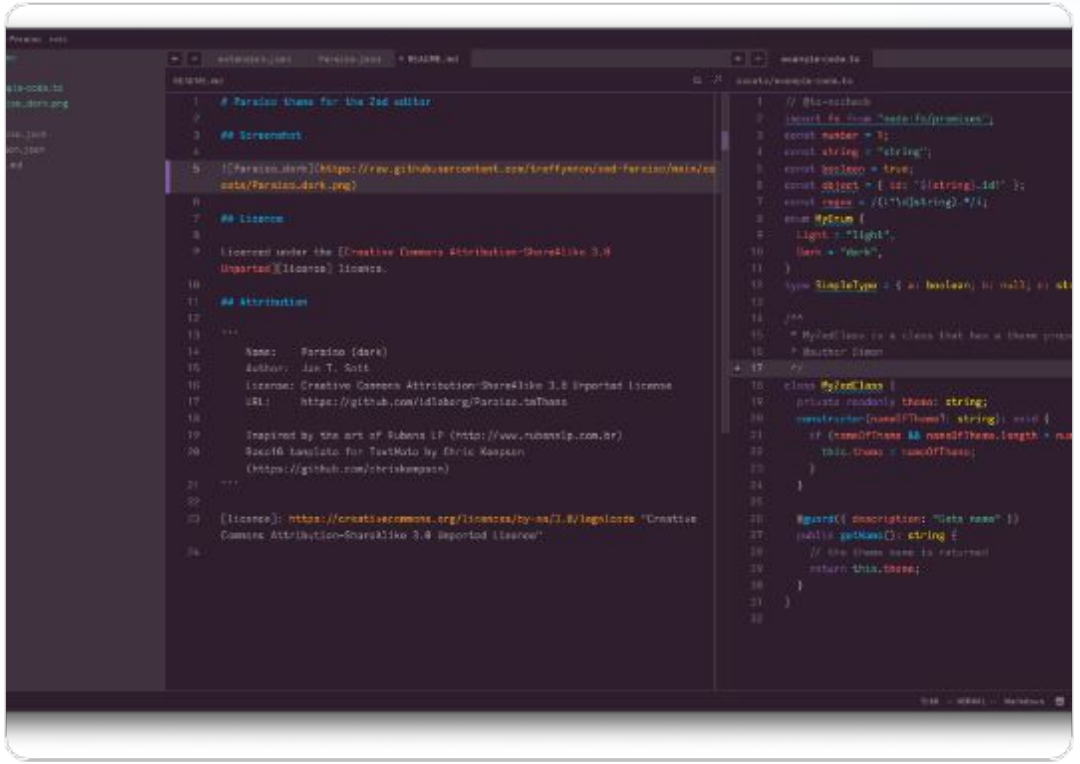
Setting Up Our Tools

Prepping the ultimate dev environment

Setting up VS Code

Visual Studio Code is a powerful, lightweight, and free code editor developed by Microsoft.

- Built-in syntax highlighting and code auto-complete.
- Supports an ecosystem of productivity extensions.



LOCUS 2027
23rd National Technological Festival

Data Hub

Setting up VS Code

WHY NOT NOTEPAD OR MS WORD???

Setting up Live Server

Never manually hit refresh again. Live Server is a crucial VS Code extension that spins up a local server and auto-reloads your page instantly whenever files are saved.

Pro-Tip

Use the shortcut **Ctrl + S** (or Cmd + S) to trigger instant web updates.

QUICK INSTALLATION STEPS

1. Open Extensions sidebar (Ctrl+Shift+X)
2. Search for "Live Server" by Ritwick Dey
3. Click the blue **Install** button
4. Right-click index.html & select **"Open with Live Server"**

Creating Our Project Folder

Clean structure prevents chaotic debugging.

Let's create a single centralized directory for all of our project's assets.

- Start simple: a single index.html file.
- We will break out styles and scripts into modern independent files later.

my-landing-page/

└── index.html

HTML — Structure

Creating the foundational architecture of the web

What is a Tag?

HTML tags are instructions wrapped in angle brackets that inform the browser how to render text, media, and structural elements.

- Most tags have an opening tag and a closing tag.
- Self-closing tags (e.g., ,) do not wrap content.

Elements and Attributes

An **element** represents the entire structure (tags + content). An **attribute** provides metadata or options inside the opening tag.

Anatomy of an Element

```
<tag attribute="value">Content</tag>
```

Attributes are always configured as key="value" pairs written into the opening tag.

Image with attributes

```
<img src="photo.jpg" alt="My profile">
```

Common HTML Tags

TAG	ELEMENT TYPE	STANDARD PURPOSE
`<h1> to <h6>`	Headers	Defines headings (h1 is the most important, h6 is lowest).
`<p>`	Paragraph	Standard text block container with automatic default spacing.
`<img>`	Image	Embeds images; requires 'src' (source) and 'alt' attributes.
`<a>`	Anchor Link	Creates links to other pages or files; requires 'href'.
`<div>`	Division	Blank container used to bundle and style related elements together.

Basic Structure of a Web Page

A standardized boilerplate that all HTML documents must adopt to render and run properly.

- </> <head>: Metadata, search-engine optimization configurations, and document styling.
- <body>: The actual visible layout and structure on your screen.

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello World</h1>
  </body>
</html>
```

Let’s Tinker Some Code

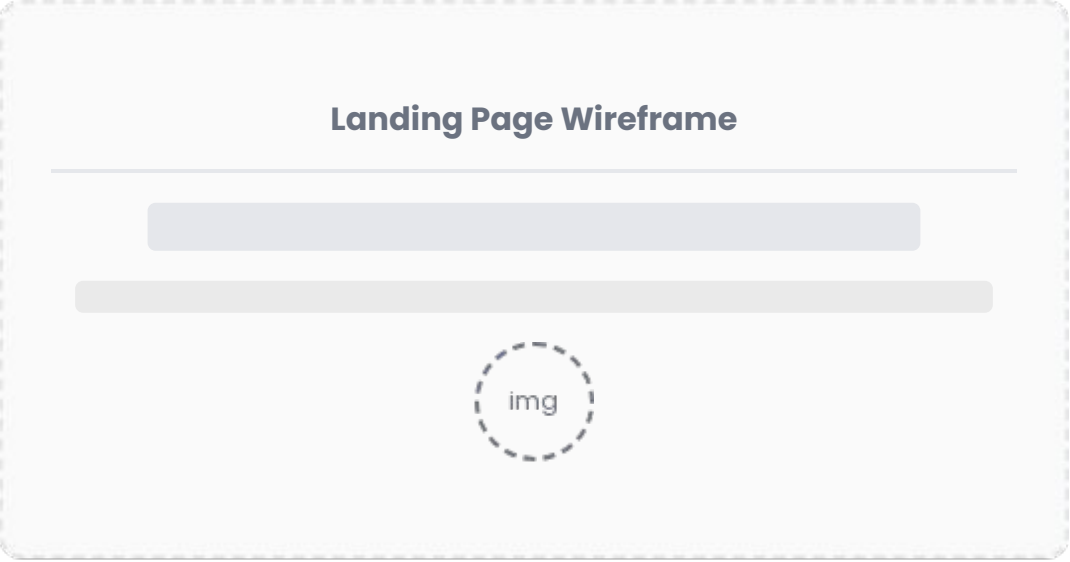
Let’s go to **VS Code**

Let's Practice

Build the skeleton of your landing page — add a primary heading (), one brief paragraph about yourself, and a profile image tag.

Suggested Steps

1. Open index.html inside VS Code.
2. Type "!" and press Tab to quickly generate structural tags.
3. Inside the body, add h1, p, and img tags.
4. Fire up **Live Server** to verify.



CSS — Styling It Up

Breathe life, color, and dynamic alignment into layouts

Adding CSS — Method 1 (Inline)

CSS stands for Cascading Style Sheets. Method 1 involves writing style code directly into HTML elements as attributes.

Drawback

Messy and difficult to scale. Use inline styles sparingly for testing single attributes.

index.html

```
style="color: blue; font-size: 20px;">
  Hello World
```

Adding CSS — Method 1b (Internal)

Internal styles are grouped inside a element nested within the HTML document's .

- ✓ Better than inline, keeping styles organized in one location.
- ⚠ Still ties style properties exclusively to a single file.

index.html

```
</div>
  p {
    color: blue;
    font-size: 20px;
  }
```

CSS Selectors

ELEMENT SELECTOR

p { ... }

Targets and styles every single matching native tag across the entire page.

CLASS SELECTOR

.card { ... }

Targets elements sharing a class attribute. Reusable on multiple elements.

ID SELECTOR

#header { ... }

Targets a unique element ID attribute. Only use once per HTML document.

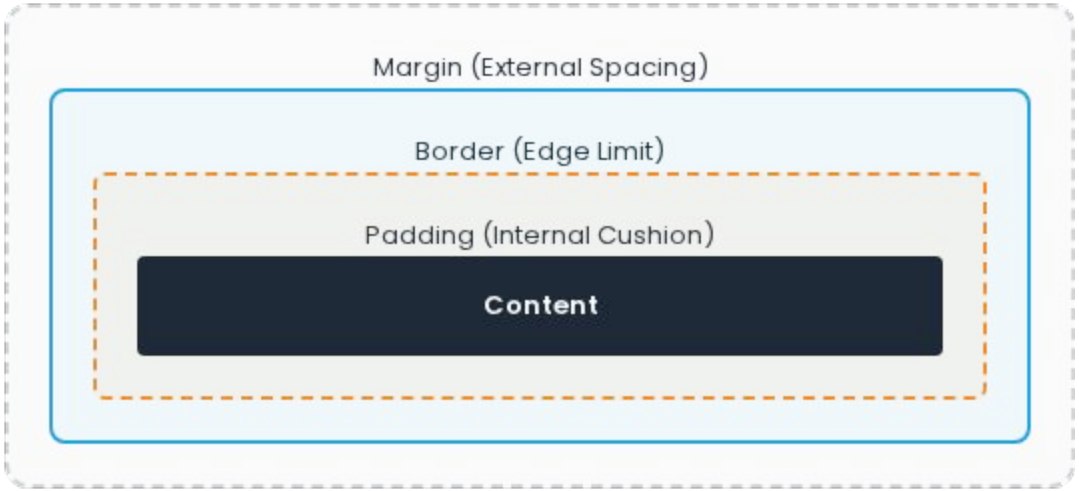
CSS Selectors

Let’s move to VS Code

The Box Model

Every element in HTML is rendered as a box. Understanding margins, borders, and paddings controls layouts accurately.

- **Content:** The text or media.
- **Padding:** Space inside the border.
- **Border:** Boundary box wrapper.
- **Margin:** Outer spacing around borders.



Flexbox for Layout

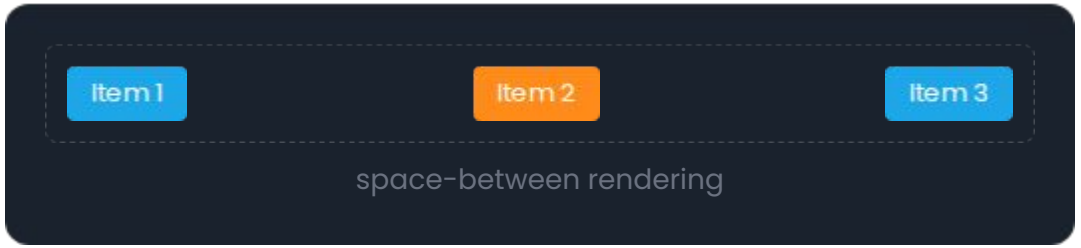
Applying display: flex to a parent container allows you to align and space child elements in rows or columns with ease.

Primary Alignment Rules

justify-content: Horizontally aligns along main axis.

align-items: Vertically aligns along cross axis.

DEMONSTRATION



Let's Practice

Style your main heading, center your landing page content, and transform your square profile image into a clean circle using the `border-radius` property.

Step-by-step styling tips:

- Target standard tags: body, h1, img.
- Add text-align: center; inside the body tag.
- Set border-radius: 50%; on your profile picture.
- Cap image width/height (e.g. 150px) to prevent layout distortion.

Buttons, Cards, Forms & Navbars

Mastering core layout patterns of modern web products

Building a Navbar

A navigation bar is typically designed as a horizontal list of links wrapped in semantic markup.

- Apply display: flex to position items in a row.
- Use list-style: none to hide bullet points.

HTML & CSS

```
    href="#">Home


/* CSS */
nav ul { display: flex; justify-content: space-between; }
```

Designing a Card

Cards isolate and group related media and details together, utilizing structural properties.

Key Box Properties

- border-radius: Creates smooth, rounded card corners.
- box-shadow: Projects subtle structural depth.
- padding: Prevents text content from touching the edges.

Visual Feature Card

Clean structure, depth, padding and responsive layout alignments.

Styling Buttons

Standard HTML buttons require custom CSS styling to transition from basic defaults into actionable, styled clickable targets.

- Define custom transitions for hovering interactive states.

style.css

```css
button {
   background: #FF8C1A;
   color: white;
   padding: 10px 20px;
   border-radius: 8px;
   border: none;
 }
button:hover { opacity: 0.85; }
```

Building a Form

Forms collect user feedback and contacts.

Structuring input and text-area controls

correctly keeps user validation clear.

- Configure inputs using type (e.g. text, email).

index.html

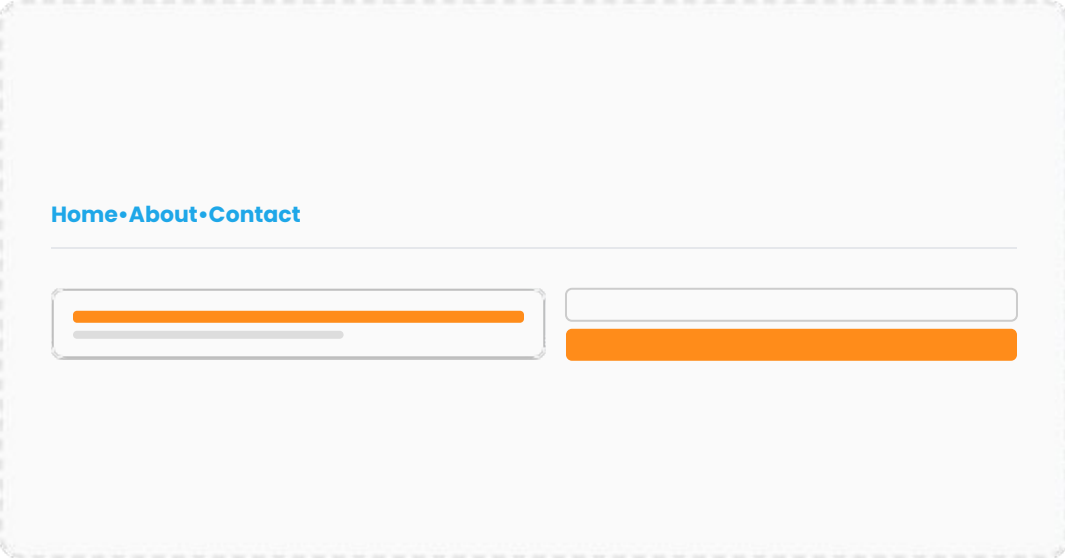
```
 type="text" placeholder="Name">
 type="email" placeholder="Email">
 placeholder="Message">
Submit
```

Let's Practice

Add a structured navbar, one about/skills card layout, and a functional contact form to your local project. Style all three using custom colors.

Implementation milestones

- Nest an anchor list inside a standard nav container.
- Wrap your skills content block inside a structured div card.
- Create inputs with appropriate styling and hover configurations.



JavaScript — Making It Interactive

Breathe logical behavior, conditions, and interaction into websites

Variables and Functions

Variables store application state and data.

Functions bundle reusable, structural blocks of execution logic.

- Variables use modern declarators: const and let.

script.js

```
const name = "Sunil";

function greet() {
   console.log("Hello " + name);
 }
```

Adding JS — Method 1 (Internal)

Method 1 involves injecting your executable JavaScript blocks directly inside standard HTML tags.

Drawback

Mixing computational logic with presentation structures becomes highly cluttered inside production frameworks.

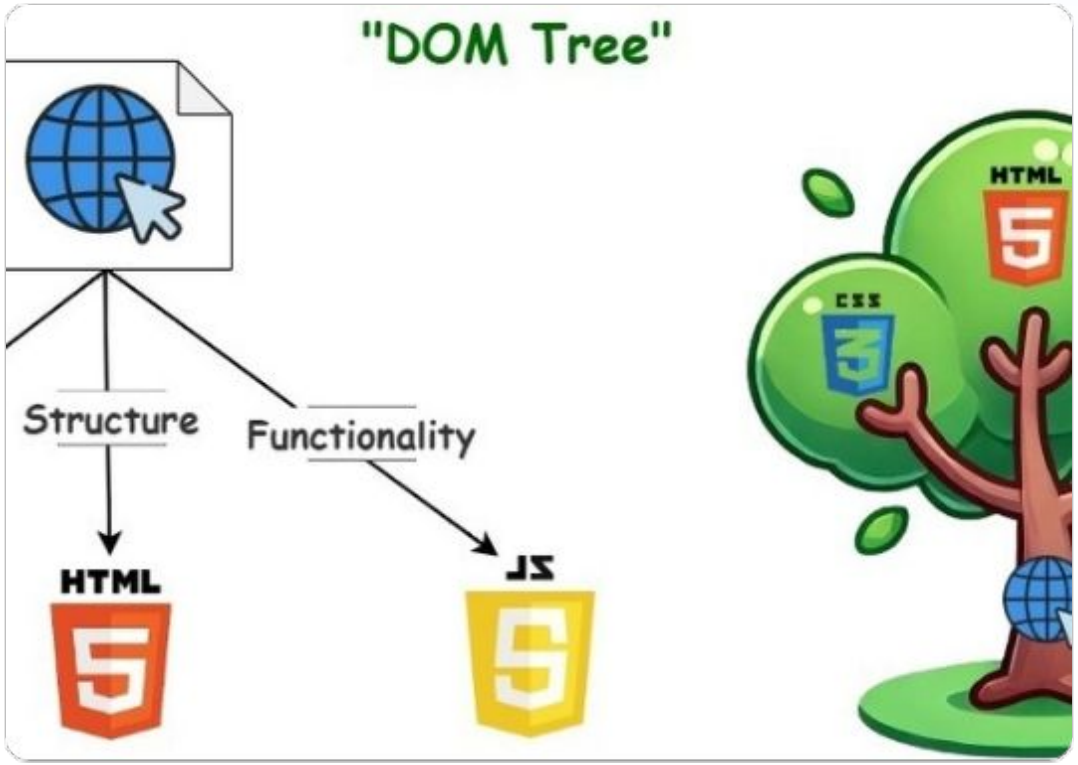
index.html

```
Hello
</div>
  console.log("Page loaded successfully!");
```

What is the DOM?

The Document Object Model (DOM) is a structured, live layout tree generated by the browser representing your HTML markup.

- JavaScript can inspect, manipulate, or rebuild nodes.
- Enables dynamic visual shifts without reload events.



Selecting Elements with JS

Before altering values or rendering events, you must inform JavaScript which exact element inside the page you want to capture.

Modern DOM Hook

document.querySelector() captures the first element matching any CSS selector parameter.

script.js

```
// Target elements by class
const btn = document.querySelector("button");

// Target sections by ID
const nav = document.querySelector("#main-nav");
```

Event Listeners

Event listeners track user activities (clicks, typing, submissions) and execute specific callback logic in response.

- 🔔 **Allows components to sit quietly until triggered.**

script.js

```
btn.addEventListener("click", function() {
   console.log("Button clicked!");
 });
```

Making the Form Interactive

Let's intercept standard submit actions to process the data locally instead of forcing a full page reload.

Essential Command

e.preventDefault() prevents form submissions from performing default page refreshes.

script.js

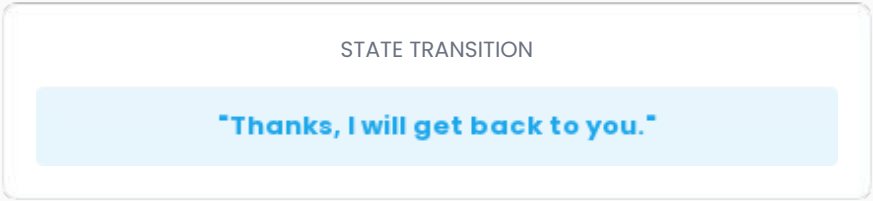
```
form.addEventListener("submit", function(e) {
  e.preventDefault();
  message.textContent = "Thanks! I'll connect soon.";
});
```

Let's Practice

Add an event listener to your form's submit button that dynamically displays a thank-you message on screen instead of reloading the page.

Implementation recipe:

1. Set an ID on your message container: .
2 Capture both form and status nodes in JS.
3. Inside the listener, change text properties.
4. Verify in browser console.



Adding CSS and JS — Method 2

The professional approach: write separate code files and link them together in your primary HTML document.

- Keeps codebase files organized, reusable, and structured.

index.html Links

```
rel="stylesheet" href="style.css">

src="script.js">
```

Setting Up the Project Folder

Establish a clean architecture directory. Create three distinct files inside one root folder.

- **index.html** holds HTML tags.
- **style.css** holds design rules.
- **script.js** holds interactive scripts.

Production Layout

The Lab

Time to build your landing page project

LOCUS 2027
23rd National Technological Festival

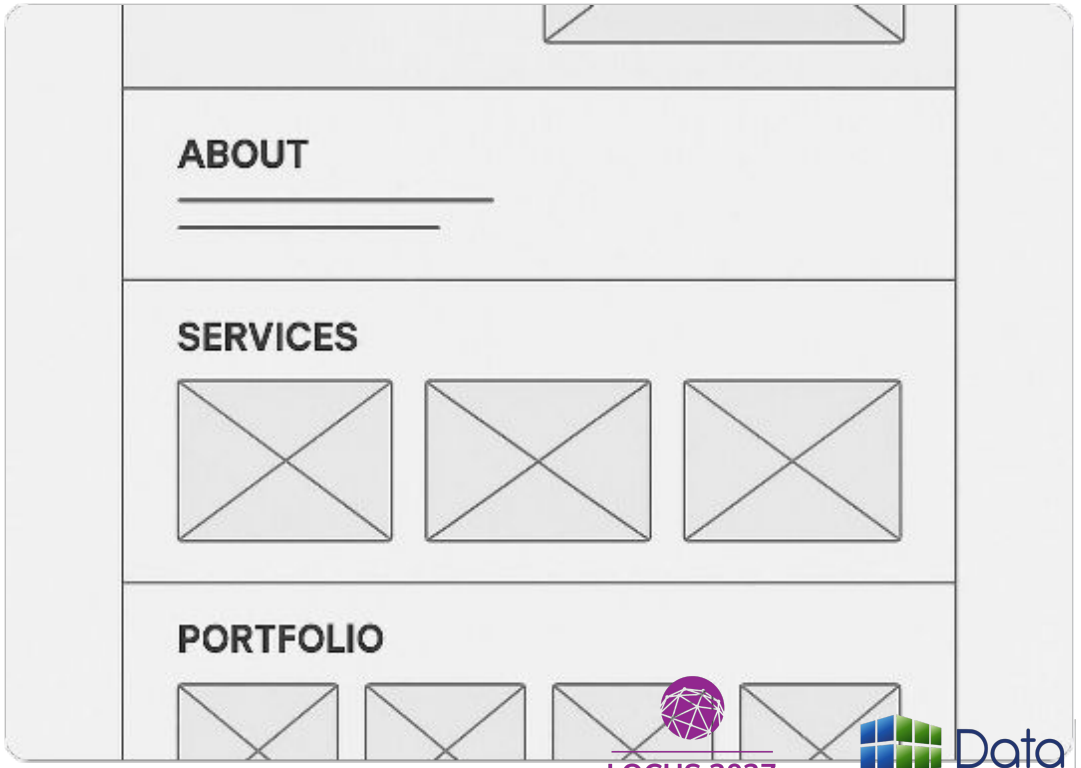
Data Hub

Today's Lab

Build the first frontend page of your project using separate HTML, CSS, and JS files. Design an attractive personal portfolio landing page containing:

Structural Deliverables

- **Navigation Bar:** Clean layout links.
- **Hero Section:** Title, brief header text.
- **Skills Card:** Boxed styled div block.
- **Contact form:** Styled submissions interface.
- **Interactive state:** Dynamic submissions message.



What We Covered Today

- Frontend vs Backend concepts
- HTML structure, tags, & elements
- VS Code editor & Live Server
- Semantic tags & SEO basics
- CSS rules & CSS box model
- Adding styles: Inline vs Separate files
- Forms, card patterns, & flexbox navbars
- JavaScript variables, DOM tree, & click events

Sunil Lamichhane

Computer Engineering Student

Toggle Dark Mode

About Me

I am a student learning frontend development, one project at a time.

Skills

- HTML
- CSS
- JavaScript

THANK YOU

Questions?

Sunil Lamichhane

LOCUS 2027
23rd National Technological Festival

Data Hub